

HOW TO FORMAT A STRING IN PYTHON

```
name = 'YoungWorks'  
string = 'Hello, %s ' % name  
string = 'Hello, ' + name  
string = 'Hello, {}'.format(name)  
string = f'Hello, {name}'  
print(string)
```

CONCATENATION

+

F STRING

f'

OLD STYLE

%

.FORMAT

{}



Hi ha diverses formes de visualitzar la informació combinant lletres i números.

3.1. Concatenar cadenes

```
1 num = 3
2 cad = "entra un número mayor que " + str(num) + ": "
3
4 num2 = input(cad)
```

entra un número mayor que 3:

3.2. f-strings

La cadena ha d'anar precedida d'una f enganxada. Les variables van entre claus { }

```
num = 3
cad = f"entra un número major que {num}:"
num2 = input(cad)
```

```
## declarem variables
nom_empresa = "Tantar"
tipus_empresa = "Educació"

## tanca amb {} el nom de la variable que vols mostrar
print(f"{nom_empresa} és una empresa de {tipus_empresa}.")
```

Tantar és una empresa de Educació.

3.3. Mostrant dades en versions anteriors de Python

#1 Format de cadena "Estil antic" (% Operador)

Les cadenes de Python tenen una operació integrada única a la qual es pot accedir amb l'operador. Això us permet fer un format posicional senzill molt fàcilment. Si alguna vegada heu treballat amb una funció `printf` style en C, reconeixeu com funciona a l'instant. Aquí teniu un exemple senzill:

Python

>>>

```
>>> 'Hello, %s' % name
"Hello, Bob"
```

3.4. Funció de cadena : format

#2 Format de cadena "Nou estil" (`str.format`)

Python 3 va introduir una [nova manera de fer el format de cadenes](#) que també es va portar posteriorment a Python 2.7. Aquest format de cadena de "nou estil" elimina la %sintaxi especial -operator i fa que la sintaxi per al format de cadena sigui més regular. El format es gestiona ara cridant `.format()` a un objecte de cadena .

Podeu utilitzar-lo `format()` per fer un format posicional senzill, tal com ho podríeu fer amb el format "estil antic":

Python

>>>

```
>>> 'Hello, {}'.format(name)
'Hello, Bob'
```

O bé, podeu fer referència a les vostres substitucions de variables pel nom i utilitzar-les en l'ordre que vulgueu. Aquesta és una característica força potent, ja que permet reordenar l'ordre de visualització sense canviar els arguments passats a `format()`:

```
name = "Bob"
errno = 30

cadena = 'Hola {a}, hi ha un error núm {b} !'.format(a=name, b=errno)
print(cadena)
```

Hola Bob, hi ha un error núm 30 !

```
name = "Bob"
errno = 30
```

```
cadena = 'Hola {}, hi ha un error núm {} !'.format(name, errno)
print(cadena)
```

Hola Bob, hi ha un error núm 30 !

```
In [1]: v = "otro texto"
        n = 10
        print("Un texto",v,"y un número",n)

Un texto otro texto y un número 10
```

Per especificar es fan servir [claus](#) en lloc de la variable,
i després la funció [.format](#) amb totes les variables separades per comes

```
In [2]: c = "Un texto {} y un número {}".format(v,n)
        c
```

```
Out[2]: 'Un texto otro texto y un número 10'
```

```
In [5]: print( "Un texto '{0}' y un número '{1}'".format(v,n) )

Un texto 'otro texto' y un número '10'
```

```
In [6]: print( "Un texto '{1}' y un número '{0}'".format(v,n) )

Un texto '10' y un número 'otro texto'
```

```
In [7]: print( "Un texto '{texto}' y un número '{numero}'".format(texto=v,numero=n) )
```

Un texto 'otro texto' y un número '10'

```
In [9]: print( "Un texto '{v}' y un número '{n}'".format(n=n,v=v) )
```

Un texto 'otro texto' y un número '10'

```
In [10]: print("{v},{v},{v}".format(v=v))
```

otro texto,otro texto,otro texto

```
In [14]: print( "{:>30}".format("palabra") ) # Alineamiento a la derecha en 30 caracteres
```

palabra

```
In [15]: print( "{:30}".format("palabra") ) # Alineamiento a la izquierda en 30 caracteres
```

palabra


```
In [16]: print( "{:^30}".format("palabra") ) # Alineamiento al centro en 30 caracteres
```

```
palabra
```

```
In [17]: print( "{:.3}".format("palabra") ) # Truncamiento a 3 caracteres
```

```
pal
```

```
In [20]: print( "{:>30.3}".format("palabra") ) # Alineamiento a la derecha en 30  
caracteres con truncamiento de 3
```

```
pal
```

P71. Práctica de format

1. Formatea los siguientes valores para mostrar el resultado indicado:

- "Hola Mundo" → Alineado a la derecha en 20 caracteres
- "Hola Mundo" → Truncamiento en el cuarto carácter (índice 3)
- "Hola Mundo" → Alineamiento al centro en 20 caracteres con truncamiento en el segundo carácter (índice 1)

3.2.3. Avançat: Formateig de xifres

```
In [23]: # Formateo de números enteros, rellenos con espacios
print("{}".format(10))
print("{}".format(100))
print("{}".format(1000))
```

10
100
1000

```
In [24]: # Formateo de números enteros, rellenos con espacios
print("{:4d}".format(10))
print("{:4d}".format(100))
print("{:4d}".format(1000))
```

10
100
1000

Reserva cuatro posiciones enteras alineadas a la derecha.

```
# Formateo de números en
print("{:04d}".format(10))
print("{:04d}".format(100))
print("{:04d}".format(1000))
```

0010
0100
1000

Reserva cuatro posiciones enteras alineadas a la derecha y rellenas con ceros.

```
In [27]: # Formateo de números flotantes, rellenos con espacios  
print("{}".format(3.1415926))
```

3.1415926

```
In [29]: # Formateo de números flotantes, r  
print("{:.3f}".format(3.1415926))
```

3.142

```
In [30]: # Formateo de números flotantes, rellenos con espacios  
print("{:.3f}".format(3.1415926))  
print("{:.3f}".format(153.21))
```

3.142

153.210

```
In [31]: # Formateo de números flotantes, rellenos con espacios  
print("{:7.3f}".format(3.1415926))  
print("{:7.3f}".format(153.21))
```

3.142

153.210

```
# Formateo de números flotantes, rel  
print("{:07.3f}".format(3.1415926))  
print("{:07.3f}".format(153.21))
```

003.142

153.210

P72. Pràctica format de números

1. Formatea los siguientes valores para mostrar el resultado indicado:

- 150 → Formateo a 5 números enteros rellenos con ceros
- 7887 → Formateo a 7 números enteros rellenos con espacios
- 20.02 → Formateo a 3 números enteros y 3 números decimales

2. Mejora la impresión de salida de este programa

```
dades = []  
dades.append(["Ciutat i estat", "Paperes", "Data del cens"])  
dades.append(["Berlín, Alemanya", "3000", "2020/12/31"])  
dades.append(["Madrid, Espanya", " 9.70", "2019/12/23"])  
dades.append(["Roma, Itàlia", "10", "2018/01/07"])  
  
for llista in dades :  
    print (llista[0], llista[1], llista[2] )
```

P73. Saldos

Programa els càlculs al quadern, copiant els enunciats com a comentari

- **Demana** 3 xifres (poden ser senceres o amb decimals, positives o negatives).
- Imagina que són entrades o sortides a un compte bancari.
- Imprimeix-les una assota de l'altre de forma que quedin ben alineades considerant el punt decimal, i mostra només el signe negatiu.
- Al costat de cada xifra imprimeix el saldo, però formatat amb zeros i que mostri sempre el signe.

Ajuda : Per fer proves, assigna valors fixes a les variables.

-3.44	-00003.44
1200.00	+01196.56
67.00	+01263.56

P74. Unidades, decenas, ...

- 1.- Pedir un número entre cero y 9999 (validar la entrada)
- 2.- Descomponer en factores de 10
 - millares = Calcular cuántos millares
 - centenas = Calcular cuántas centenas
 - decenas = Calcular cuántas decenas
 - unidades = calcular cuantas unidades
- 3.- Imprimir unidades, decenas, centenas, millares, en líneas separadas formateados del siguiente modo :
 - 000u u = unidades
 - 00d0 d= decenas
 - 0c00 c=centenas
 - m000 m= millares

Ejemplo

Introduce un número : 3647

```
0007
0040
0600
3000
```

Una solución

```
num = int(input ("Introduzca Número: "))

millares=int(num /1000)
centenas=int((num-(millares*1000))/100)
decenas=int((num- (millares*1000 + centenas*100))/10)
unidades=int(num-(millares*1000 + centenas*100 + decenas*10 ))

print (":04d".format(unidades))
print (":04d".format(decenas * 10))
print (":04d".format(centenas * 100))
print (":04d".format(millares * 1000) )
```

Ejemplo

Introduce un número : 3647

0007

0040

0600

3000